Árboles de Decisión

Limitaciones de los Modelos Lineales

Los modelos lineales (como la regresión lineal o logística) asumen una relación lineal entre las variables predictoras y la variable objetivo:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Problemas:

- No capturan interacciones complejas entre variables
- Requieren transformaciones manuales de features (feature engineering)
- Sensibles a outliers y multicolinealidad

Necesidad de Modelos Más Flexibles

En muchos problemas reales, las relaciones entre variables no son lineales:

- **Ejemplo:** El precio de una casa no aumenta linealmente con el tamaño
- **Ejemplo:** La probabilidad de churn no depende linealmente de la antigüedad del cliente

Los árboles de decisión ofrecen una alternativa flexible que puede:

- Capturar relaciones no lineales automáticamente
- Manejar interacciones entre variables sin especificarlas explícitamente

¿Qué es un Árbol de Decisión?

Un árbol de decisión es una estructura jerárquica que representa un conjunto de decisiones y sus posibles consecuencias.

Componentes principales:

- **Nodo raíz:** Primera decisión, basada en la variable más importante
- **Nodos internos:** Decisiones intermedias basadas en otras variables
- **Hojas (nodos terminales):** Resultados finales o predicciones

Cómo funciona:

- Se divide el conjunto de datos en **subconjuntos cada vez más homogéneos**
- Cada división se basa en una **condición sobre una variable predictora**
- El proceso continúa hasta que se alcanza un **criterio de parada**

Ejemplo: Árbol para Decidir si Jugar al Tenis

```
¿Está soleado?
├── Sí → ¿Menos de 30 °C?
│    ├── Sí → No jugar
│    └── No → ¿Hay viento?
│         ├── Sí → No jugar
│         └── No → Jugar
└── No → ¿Llueve?
     ├── Sí → No jugar
     └── No → Jugar
```

Interpretación:

- Si está soleado Y menos de 30 °C Y no hace viento → Jugar
- Si está soleado Y más de 30 °C → No jugar
- Si no está soleado y llueve → No jugar
- Etc.

Criterios de División en Regresión

Criterio habitual: minimizar error cuadrático medio (mismo que la regresión lineal)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$$

Mientras no se cumpla criterio de parada repetimos los siguientes pasos:

- Para cada combinación de hoja, variable predictora X y posible punto de división p:
  i. Se divide las n observaciones del la hoja en dos grupos: $x_i \leq d$ vs $x_i > d$
  - Si X es categórica, se consideran todas las particiones posibles de las categorías
  ii. Se calcula el MSE total para esa combinacion
- Se elige la combibinación con menor MSE total
  - Se divide la hoja de esa combinación en dos nuevas hojas
  - En cada hoja promedia la variable objetivo $\bar{y}$ y se asigna como valor de predicción

Criterios de División en Clasificación

Si en un nodo hay 3 obs. con $y = 0$ y 2 con $y = 1$, entonces $p_0 = 0.6$ y $p_1 = 0.4$.

Índice de Gini:

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Donde p_i es la proporción de observaciones de la clase i en el nodo.

- Gini = 0: Nodo puro (una sola clase)
- Gini = 0.5: Máxima impureza (dos clases equiprobables)

Entropía (aka log loss, el mismo de la regresión logística)

$$Entropía = -\sum_{i=1}^{c} p_i \log_2 p_i$$

Ejemplo de Cálculo: Índice de Gini

Supongamos un nodo con 100 observaciones:

- 60 de clase A
- 40 de clase B

Gini del nodo padre:

$$Gini = 1 - (0.6)^2 - (0.4)^2 = 1 - 0.36 - 0.16 = 0.48$$

Después de una división:

- Nodo izquierdo: 30 A, 10 B → $Gini = 1 - (0.75)^2 - (0.25)^2 = 0.375$
- Nodo derecho: 30 A, 30 B → $Gini = 1 - (0.5)^2 - (0.5)^2 = 0.5$

Gini ponderado:

$$Gini_{total} = (40/100) \times 0.375 + (60/100) \times 0.5 = 0.15 + 0.3 = 0.45$$

Hiperparámetros en Árboles de Decisión

Los hiperparámetros controlan el crecimiento del árbol y afectan al tradeoff entre bias y varianza (underfitting vs overfitting).

Hiperparámetros principales:

- **max_depth:** Profundidad máxima del árbol
  - Valores típicos: 3-10
  - Mayor profundidad → Más riesgo de overfitting
- **min_samples_split:** Mínimo de observaciones para dividir un nodo
  - Valores típicos: 2-10
  - Mayor valor → Menos overfitting, pero posible underfitting

Hiperparámetros principales: (continuación)

- **min_samples_leaf:** Mínimo de observaciones en una hoja
  - Valores típicos: 1-5
  - Similar a min_samples_split, controla el tamaño mínimo de las hojas resultantes
- **max_features:** Número máximo de variables consideradas en cada división
  - Valores: "sqrt", "log2", o número entero
  - Reduce correlación entre árboles en Random Forest
- **criterion:** Criterio de división
  - Clasificación: "gini" o "entropy"
  - Regresión: "mse" o "mae"

Ventajas: Menos Feature Engineering

Feature Engineering Tradicional:

- Crear variables polinomiales: x^2, x^3
- Interacciones manuales: $x_1 \times x_2$
- Transformaciones: logaritmos, raíces cuadradas
- Discretización de variables continuas

Con Árboles de Decisión:

- El algoritmo encuentra automáticamente las divisiones óptimas
- No requiere conocimiento experto del dominio
- Maneja variables categóricas y numéricas sin preprocesamiento

Tradeoff: flexibilidad vs Overfitting

Flexibilidad:

- Los árboles pueden capturar relaciones complejas en los datos
- Mayor profundidad permite modelar interacciones y no linealidades

Overfitting en Árboles:

- Los árboles pueden crecer indefinidamente, capturando ruido en los datos de entrenamiento
- Cada división adicional mejora el ajuste en training, pero puede empeorar en test

Desventajas: Menos Interpretabilidad

Modelos Lineales:

- Coeficientes claros: β_i indica el impacto de cada variable
- Fácil explicar: "Por cada unidad que aumenta x, y cambia en β unidades"
- Extrapolación: el comportamiento del modelo se extiende fuera del rango de los datos

Árboles de Decisión:

- Reglas de decisión complejas
- Difícil cuantificar el impacto global de una variable
- Interpretación requiere visualizar el árbol completo
- Extrapolación: observaciones fuera del rango van a las mismas hojas que las observaciones dentro del rango

Mitigación: Importancia de las Variables (Feature Importance)

En los árboles de decisión, podemos medir la importancia de cada variable predictora basándonos en cuánto contribuye a la mejora de la predicción en el árbol.

Cómo se calcula:

- Para cada variable, se define la importancia como la suma de la mejora de la predicción ponderada por el número de observaciones en cada nodo donde se usa esa variable
- Se normaliza para que la suma de todas las importancias sea 1

Interpretación:

- Variables con mayor importancia tienen mayor impacto en las predicciones
- Útil para selección de features y comprensión del modelo

Métodos Ensemble

Combinaciones de modelos débiles para crear un modelo más fuerte y robusto.

Principios fundamentales:

- **Bagging (Bootstrap Aggregating):** Reduce la varianza combinando modelos entrenados en subconjuntos aleatorios de datos
- **Boosting:** Reduce el bias entrenando modelos secuencialmente, cada uno corrigiendo los errores del anterior
- **Objetivo:** Mejorar la generalización y reducir overfitting

Ventajas de los ensembles:

- Mejor rendimiento que modelos individuales
- Mayor estabilidad y robustez
- Menos sensibles a hiperparámetros

Random Forest: Bagging con Árboles

Random Forest construye múltiples árboles de decisión independientes y combina sus predicciones mediante votación o promedio.

Algoritmo:

1. **Bootstrapping:** Crea múltiples subconjuntos de datos mediante muestreo con reemplazo
2. **Feature Randomness:** Cada árbol usa un subconjunto aleatorio de variables predictoras
3. **Construcción:** Entrena un árbol de decisión en cada subconjunto
4. **Agregación:** Para clasificación: mayoría de votos; para regresión: promedio de predicciones

Random Forest: Hiperparámetros

Hiperparámetros clave:

- Número de árboles (n_estimators)
- Profundidad máxima de cada árbol
- Número de features por división

Ventajas de Random Forest

- **Robustez:** Menos propenso a overfitting que un solo árbol
- **Manejo de datos:** Funciona bien con datos mixtos (numéricos y categóricos)
- **Paralelización:** Los árboles se pueden entrenar en paralelo
- **Out-of-bag error:** Estimación automática del error sin validación cruzada

Gradient Boosting: Boosting con Árboles

Gradient Boosting construye árboles secuencialmente, donde cada árbol corrige los errores del modelo anterior.

Algoritmo:

1. **Modelo inicial:** Comienza con una predicción simple (ej. media de la variable objetivo)
2. **Cálculo de residuos:** Calcula los errores (gradientes) del modelo actual
3. **Entrenamiento:** Entrena un nuevo árbol para predecir los residuos
4. **Actualización:** Actualiza el modelo sumando el nuevo árbol multiplicado por un learning rate pequeño
5. **Iteración:** Repite los pasos 2-4 hasta convergencia o límite de iteraciones

Gradient Boosting: Hiperparámetros

Hiperparámetros clave:

- Learning rate (tasa de aprendizaje)
- Número de iteraciones (n_estimators)
- Profundidad máxima de los árboles

Ventajas de Gradient Boosting

- **Alta precisión:** A menudo supera a otros algoritmos en competiciones
- **Flexibilidad:** Maneja relaciones no lineales y complejas
- **Regularización:** Incluye técnicas para evitar overfitting
- **Feature engineering automático:** Captura interacciones automáticamente

Implementaciones populares:

- XGBoost: Optimizado para velocidad y rendimiento
- LightGBM: Eficiente con grandes datasets
- CatBoost: Maneja bien variables categóricas

Comparación: Random Forest vs Gradient Boosting

Random Forest

Prioridad: robustez

- Paralelizable (entrenamiento rápido)
- Menos propenso a overfitting
- Buena interpretabilidad (importancia de variables)
- Funciona bien "out-of-the-box"

Gradient Boosting

Prioridad: máxima precisión

- Entrenamiento secuencial (más lento)
- Mayor riesgo de overfitting sin tuning
- Menos interpretable
- Requiere optimización de hiperparámetros
- Mayor precisión potencial

Como entrenar árboles de decisión

Ajuste de hiperparámetros

- Comenzar con valores conservadores (árboles pequeños)
  - Limitar la profundidad máxima del árbol
  - Evitar dividir nodos con pocas observaciones
  - Evitar dividir nodos muy homogéneos
- Usar validación cruzada para optimizar
- Grid search o random search para hiperparámetros